

Aula 30 - Streaming de Dados com SSE e WebSockets

Duração: 4 horas (2h teórica + 2h prática)

Parte Teórica (1 hora)

1. Comparação SSE vs WebSockets

Característica	Server-Sent Events (SSE)	WebSockets
Protocolo	HTTP (simples)	WS (bidirecional)
Direção	Apenas servidor → cliente	Bidirecional
Reconexão	Automática	Manual
Suporte a Dados	Apenas texto	Binário e texto
Casos de Uso	Notificações, atualizações em tempo real	Chat, jogos multiplayer

2. Quando Usar Cada Tecnologia

- **SSE:**
 - Ideal para dashboards, atualizações de status (ex: preços de ações).
 - Mais simples de implementar (não requer protocolo especial).
- **WebSockets:**
 - Necessário quando cliente e servidor precisam se comunicar ativamente (ex: chat).

3. Fluxo Básico

SSE (Spring WebFlux)

```
java

@GetMapping(path = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<String> streamData() {
    return Flux.interval(Duration.ofSeconds(1))
        .map(sequence -> "Evento " + sequence);
}
```

WebSockets (Vue.js + Spring)

javascript

```
const socket = new WebSocket("ws://localhost:8080/ws");
socket.onmessage = (event) => console.log(event.data);
```

Parte Prática (3 horas)

Atividade 1: SSE com Spring WebFlux e Vue.js (1h30min)

Backend (Spring WebFlux)

1. Endpoint de Streaming:

java

```
@RestController
public class SSEController {
    @GetMapping(value = "/api/sse/temperatura", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<Integer> streamTemperatura() {
        return Flux.interval(Duration.ofSeconds(1))
            .map(i -> new Random().nextInt(40)); // Simula sensor de temperatura
    }
}
```

2. Configuração de CORS (opcional):

java

```
@Bean
public WebFilter corsFilter() {
    return (exchange, chain) -> {
        exchange.getResponse().getHeaders().add("Access-Control-Allow-Origin", "*");
        return chain.filter(exchange);
    };
}
```

Frontend (Vue.js)

1. Conexão SSE (useSSE.js composável):

javascript

```
import { ref, onMounted } from 'vue';
```

```

export function useSSE(url) {
  const data = ref(null);
  const error = ref(null);

  onMounted(() => {
    const eventSource = new EventSource(url);

    eventSource.onmessage = (event) => {
      data.value = event.data;
    };

    eventSource.onerror = (err) => {
      error.value = err;
      eventSource.close();
    };
  });

  return { data, error };
}

```

2. Uso no Componente (Temperatura.vue):

```

vue

<template>
  <div>Temperatura atual: {{ temperatura }}°C</div>
</template>

<script setup>
import { useSSE } from '@composables/useSSE';
const { data: temperatura } = useSSE('http://localhost:8080/api/sse/temperatura');
</script>

```

Atividade 2: WebSockets com STOMP (1h30min)

Backend (Spring WebFlux + WebSocket)

1. Configuração (WebSocketConfig.java):

```

java

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
  @Override
  public void configureMessageBroker(MessageBrokerRegistry config) {
    config.enableSimpleBroker("/topic");
    config.setApplicationDestinationPrefixes("/app");
  }
}

```

```

    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws").setAllowedOrigins("*");
    }
}

```

2. Controller de Mensagens (ChatController.java):

java

```

@Controller
public class ChatController {
    @PostMapping("/chat.send")
    @SendTo("/topic/public")
    public String sendMessage(String message) {
        return "Usuário disse: " + message;
    }
}

```

Frontend (Vue.js + SockJS)

1. Instalação:

bash

```
npm install sockjs-client @stomp/stompjs
```

2. Conexão (useWebSocket.js composable):

javascript

```

import { ref } from 'vue';
import { Client } from '@stomp/stompjs';

export function useWebSocket() {
    const messages = ref([]);
    const client = new Client({
        brokerURL: 'ws://localhost:8080/ws',
        onConnect: () => {
            client.subscribe('/topic/public', (message) => {
                messages.value.push(message.body);
            });
        }
    });

    client.activate();

    const sendMessage = (text) => {
        client.publish({ destination: '/app/chat.send', body: text });
    };
}

```

```
};

return { messages, sendMessage };
}
```

3. Componente de Chat (ChatRoom.vue):

```
vue

<template>
  <div>
    <ul>
      <li v-for="(msg, index) in messages" :key="index">{{ msg }}</li>
    </ul>
    <input v-model="newMessage" @keyup.enter="sendMessage(newMessage)">
  </div>
</template>
```

Tarefa para Casa

1. Extensão SSE:

- Adicionar eventos personalizados (`event: update`) no backend e filtrar no frontend.

2. Segurança WebSocket:

- Integrar autenticação JWT nas conexões WebSocket.

Próxima Aula

- **Tópico:** Projeto Final (Vue.js + WebFlux).
- **Atividade prática:** Sistema de monitoramento em tempo real.

Resultado Esperado

- **SSE:**
 - Frontend exibindo dados em tempo real do backend.
- **WebSockets:**
 - Chat funcional com mensagens bidirecionais.

plaintext

- ✓ SSE para streaming unidirecional
- ✓ WebSockets para comunicação bidirecional
- ✓ Pronto para o projeto final!

https://miro.medium.com/v2/resize:fit:1400/1*HXGZhRkG0Lf8YlrGJ1HI4A.png (*Comparação entre tecnologias de tempo real*) 🚀